॥वसुधैव कुटुम्बकम्॥

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category – I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 05

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	**Yash Patel**
PRN No.	**23070122235**
Branch	CSE, Batch (2023-27), L4
Academic Year & Semester	25 – 26, Sem VII
Date of Performance	16th July, 2026
Title of Assignment:	Continuous Integration with Jenkins
Theory:	**Theory** Jenkins is an open-source automation server used to implement Continuous Integration (CI) and Continuous Delivery (CD). It automates software development tasks such as building, testing, and deploying applications. Jenkins integrates with version control systems like Git to automatically retrieve source code and execute predefined build steps. In this experiment, Jenkins was installed using Docker, configured with the Git plugin, and connected to a GitHub repository. A Freestyle project was created to pull source code from the repository and execute shell commands. The build was then triggered manually, and the console output was examined to verify successful execution. **Commands Used** • docker --version – Verified that Docker was installed on the system. • docker ps – Checked whether the Jenkins Docker container was running. • docker run -d -p 8080:8080 -p 50000:50000 -v jenkins_home:/var/jenkins_home --name myjenkins jenkins/jenkins:lts-jdk17 – Created and started the Jenkins Docker container. • docker logs myjenkins – Displayed the Jenkins logs to obtain the initial administrator password during the initial setup.

	• docker exec myjenkins cat /var/jenkins_home/secrets/initialAdminPassword – Retrieved the initial administrator password generated by Jenkins. **Execute Shell Commands** • echo "BUILDING: ${JOB_NAME}" – Displayed the Jenkins job name. • echo "BUILD NUMBER: ${BUILD_NUMBER}" – Displayed the current build number. • echo "WORKSPACE: ${WORKSPACE}" – Displayed the Jenkins workspace path. • ls -la – Listed all files present in the Jenkins workspace. • echo "Hello Jenkins!" – Displayed a sample message to verify successful execution of the build step.
Implementation:	**Step 1: Verify Docker and Jenkins Container** Verified Docker and Jenkins Container — Verified Docker installation and confirmed that the Jenkins Docker container (myjenkins) was running successfully. **Step 2: Access Jenkins Dashboard** Accessed Jenkins Dashboard — Successfully accessed the Jenkins Dashboard and verified that the existing Freestyle project (Git-Practice-Build) was available. **Step 3: Verify Git Plugin Installation** Verified Git Plugin — Verified that the Git plugin was installed and enabled, allowing Jenkins to integrate with Git repositories for source code management. **Step 4: Open the Freestyle Project** Opened Freestyle Project — Opened the existing Freestyle project (Git-Practice-Build) to verify its configuration and prepare it for executing the build process. **Step 5: Verify Source Code Management (Git)** Configured Source Code Management — Configured the Freestyle project to retrieve source code from the GitHub repository by selecting Git as the Source Code Management option and specifying the repository URL. **Step 6: Verify Build Step** Configured Build Step — Verified the Execute shell build step containing shell commands to display build information, list the repository files, and print a sample message during the build process. **Step 7: Save and Run the Build**

Executed Build — Successfully executed the Freestyle project using Build Now and verified that the build completed successfully with a SUCCESS status.

Step 8: View Console Output

Verified Console Output — Examined the console output and verified that Jenkins successfully cloned the Git repository, executed the configured shell commands, and completed the build with a SUCCESS status.

```
C:\Users\yashp>docker --version
Docker version 29.6.2, build dfc4efb

C:\Users\yashp>docker ps
failed to connect to the docker API at npipe:////./pipe/dockerDesktopLinuxEngine; check if the path is correct and if th
e daemon is running: open //./pipe/dockerDesktopLinuxEngine: The system cannot find the file specified.

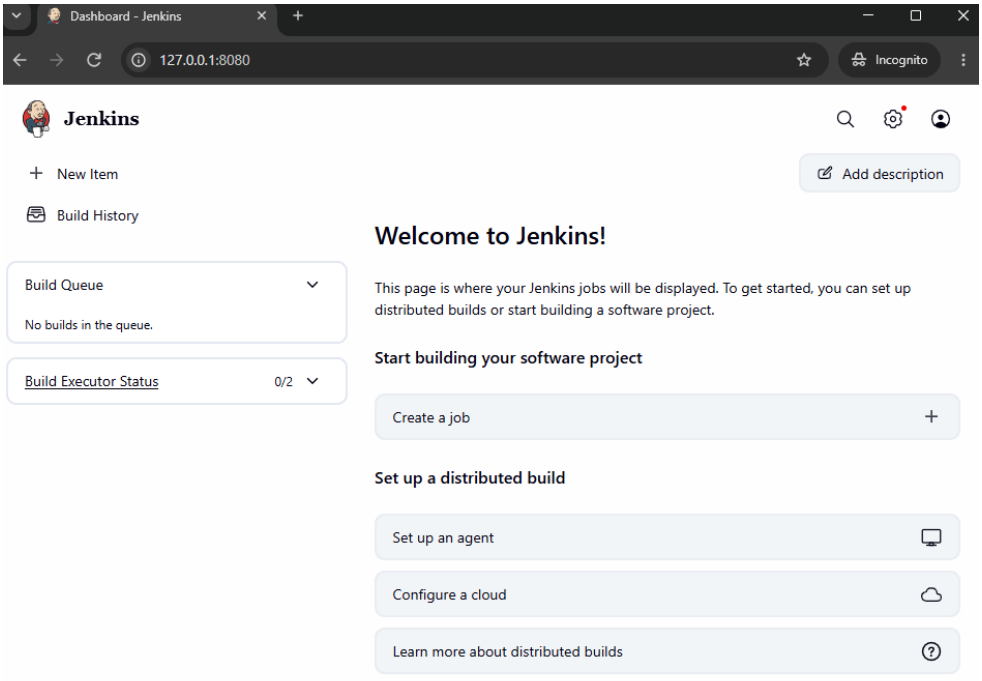
C:\Users\yashp>docker ps
CONTAINER ID   IMAGE         COMMAND                  CREATED        STATUS          PORTS
          NAMES
6469f5fb413a   postgres:15   "docker-entrypoint.s…"   2 months ago   Up 22 seconds   0.0.0.0:5432->5432/tcp, [::]:5432->
5432/tcp   flaskapp-db-1
```

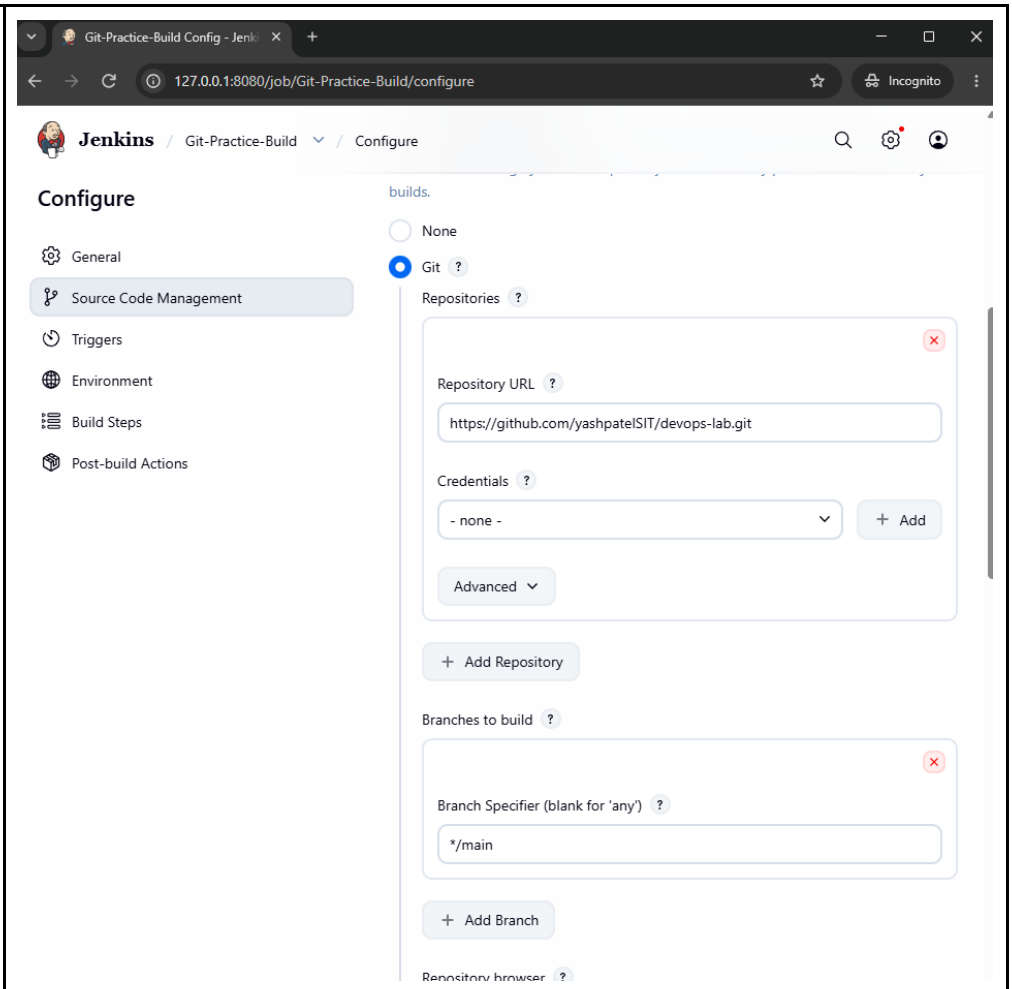
```
C:\Users\yashp>docker ps
CONTAINER ID   IMAGE                     COMMAND                  CREATED          STATUS          PORTS
                                                  NAMES
4574c5c395ed   jenkins/jenkins:lts-jdk17   "/usr/bin/tini -- /u…"   34 seconds ago   Up 33 seconds   0.0.0.0:8080->8080/
tcp, [::]:8080->8080/tcp, 0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp   myjenkins
6469f5fb413a   postgres:15               "docker-entrypoint.s…"   2 months ago     Up 3 minutes    0.0.0.0:5432->5432/
tcp, [::]:5432->5432/tcp                                         flaskapp-db-1

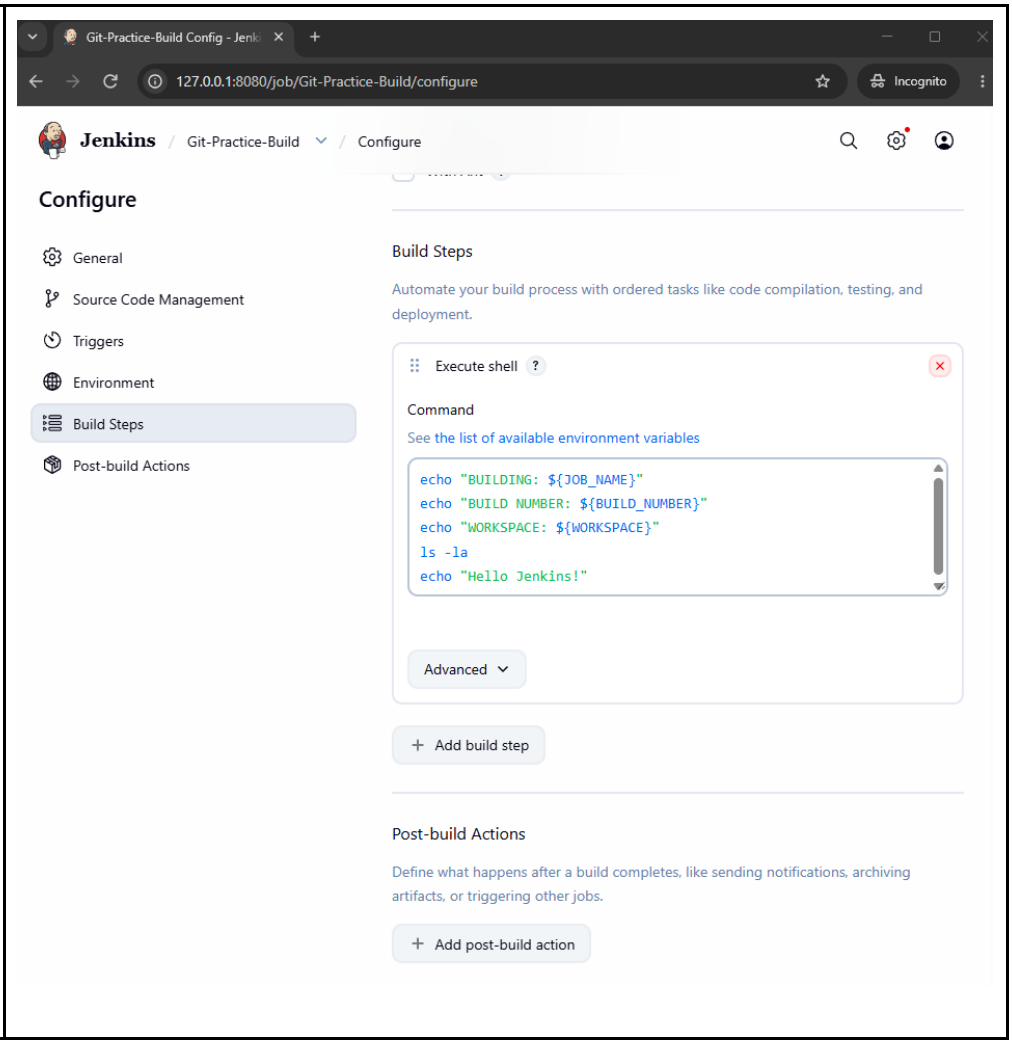
C:\Users\yashp>
```

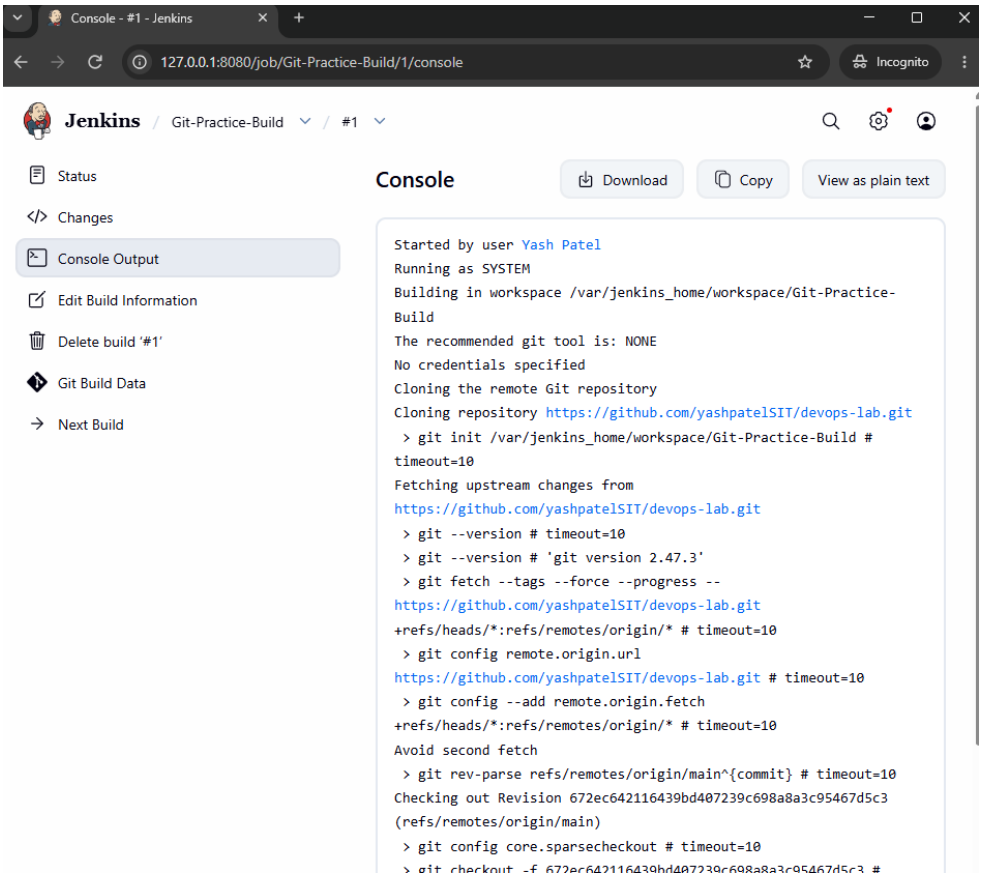
```
C:\Users\yashp>docker exec myjenkins cat /var/jenkins_home/secrets/initialAdminPassword
dd289798967f42a4a968b9c8d63ac841

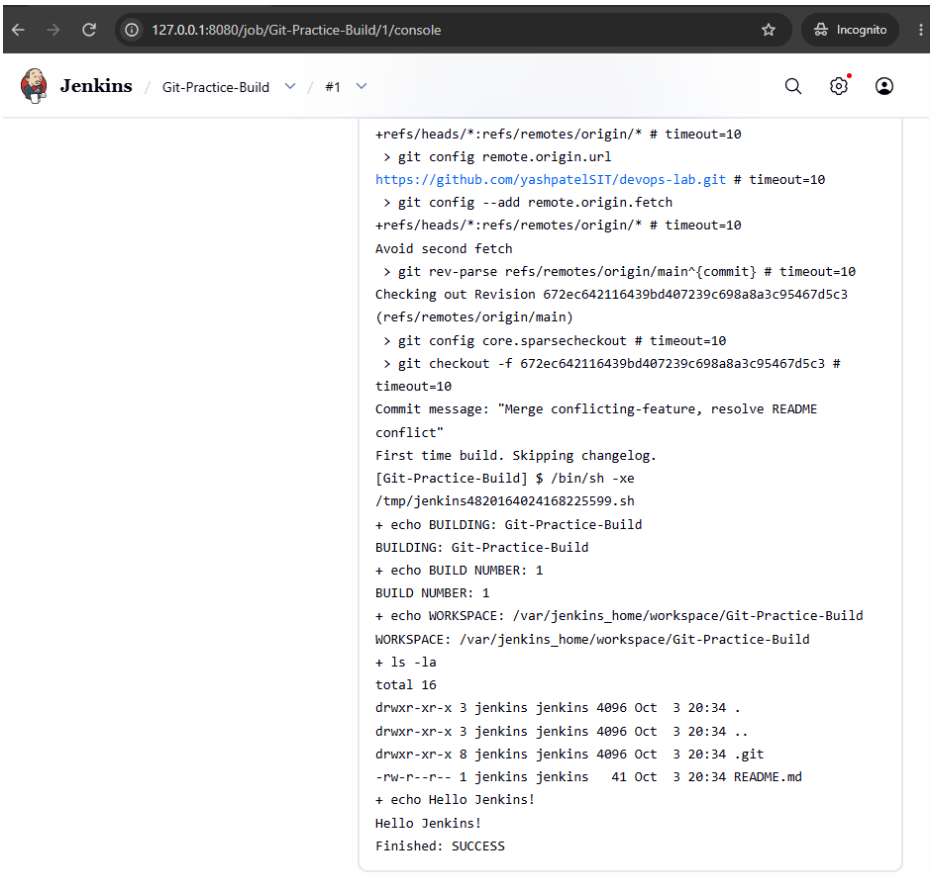
C:\Users\yashp>
```




Git-Practice-Build Config - Jenk
127.0.0.1:8080/job/Git-Practice-Build/configure
Incognito
Jenkins / Git-Practice-Build / Configure
Configure
General
Source Code Management
Triggers
Environment
Build Steps
Post-build Actions
builds.
None
Git
Repositories
Repository URL
https://github.com/yashpatelSIT/devops-lab.git
Credentials
- none -
Add
Advanced
Add Repository
Branches to build
Branch Specifier (blank for 'any')
*/main
Add Branch
Repository browser


Git-Practice-Build Config - Jenki
127.0.0.1:8080/job/Git-Practice-Build/configure
Incognito
Jenkins / Git-Practice-Build / Configure
Configure
General
Source Code Management
Triggers
Environment
Build Steps
Post-build Actions
Build Steps
Automate your build process with ordered tasks like code compilation, testing, and deployment.
Execute shell
Command
See the list of available environment variables
echo "BUILDING: ${JOB_NAME}"
echo "BUILD NUMBER: ${BUILD_NUMBER}"
echo "WORKSPACE: ${WORKSPACE}"
ls -la
echo "Hello Jenkins!"
Advanced
Add build step
Post-build Actions
Define what happens after a build completes, like sending notifications, archiving artifacts, or triggering other jobs.
Add post-build action





Subject Faculty Name: Prof. Arijit Dutta